

Damas IA: Arquitectura, Motor y Agente Reactivo

Memoria Técnica de Construcción, Decisiones de Diseño y Guía de Modificaciones

Proyecto: Damas con Inteligencia Artificial

Ciclo: Segundo Semestre 2026

Stack: TypeScript, Phaser 3, Vite

Estado: Sprint 1 Finalizado (Reglas + Agente Reactivo)

1. Principio Fundamental: Desacoplamiento Phaser vs. Motor

Desde la concepción del proyecto se estableció una distinción rigurosa: **Phaser no sabe jugar damas**. Phaser es exclusivamente la capa de visualización e interacción de entrada/salida (I/O). Toda la lógica formal del juego reside en un motor agnóstico en TypeScript puro, sin referencias a Phaser.

«El motor calcula qué es legal; el agente decide qué jugada prefiere; Phaser simplemente refleja el estado resultante y recibe clics.»

Esta separación permite que tanto el agente reactivo como futuros algoritmos (Minimax) evalúen partidas sin necesidad de simular sprites o interfaces gráficas, consumiendo únicamente el estado del juego (GameState).

2. Módulos y Responsabilidades del Sistema

Capa / Archivo	Responsabilidad Primaria
core/Piece.ts	Tipos de jugador ('human' 'ai') y pieza (player, king: boolean).
core/Move.ts	Coordenadas from, to y ficha capturada opcional (captured).
core/Board.ts	Matriz 8x8, verificación de bordes, coronaciones, eliminación y clonación profunda.
core/GameState.ts	Controlador del estado: tablero activo, turno (currentPlayer) y cálculo de ganador.
rules/CheckersRules.ts	Movimientos simples, capturas, toma obligatoria global y reyes bidireccionales.
agents/Agent.ts	Interfaz polimórfica base: chooseMove(gameState): Move null.
agents/ReactiveAgent.ts	Agente basado en reglas condición-acción (Captura > Corona > Amenaza > Avance).
game/scenes/GameScene.ts	Escena Phaser: interacción con mouse, orquestación física y temporizadores visibles.
game/board/BoardRenderer.ts	Renderizado de casillas, fichas, coronas (■) y recuadros verdes de resaltado.

3. Cronología de Reglas y Mecánicas Implementadas (Sprint 1)

- Movimiento Diagonal y Tablero 8x8:** Solo se juega sobre casillas oscuras $(row + column) \% 2 \neq 0$. Fichas normales avanzan 1 casilla diagonal.
- Capturas y Eliminación:** Salto de 2 casillas sobre pieza enemiga. Al completarse, la ficha es retirada de la matriz mediante `board.removePiece()`.
- Toma Obligatoria Global (hasAnyCapture):** Si cualquier ficha del jugador en turno puede capturar, se bloquean todos los movimientos simples del jugador. No se requiere lógica en la vista: el motor devuelve únicamente las capturas legales.

- **Captura Múltiple Encadenada:** Tras un salto, se evalúa `getCapturesForPiece` sobre la nueva coordenada. Si hay saltos subsiguientes, se asigna `forcedPiece` y se retiene el turno en la misma ficha.
- **Coronación y Reyes (`king: boolean`):** Al llegar a la fila opuesta (0 para blanco, 7 para negro), la pieza corona (■) y puede desplazarse o capturar en las 4 diagonales. Si corona en un salto, la jugada termina ahí por regla explícita.
- **Fin de Partida (`getWinner`):** Se pierde por aniquilación total de fichas (`countPieces === 0`) o por bloqueo/ahogado (`getAllValidMoves().length === 0`).
- **Pausas Visuales para la IA (500 ms):** Se integró `this.time.delayedCall(500, ...)` en Phaser para que las decisiones y saltos múltiples de la computadora sean visibles y comprensibles para el usuario.
- **Clonación de Tableros (`Board.clone`):** Duplicación profunda de celdas e instancias de piezas para sustentar la simulación de estados hipotéticos en Minimax sin alterar la partida real.

4. Arquitectura Interna del Agente Reactivo Simple

El `ReactiveAgent` satisface estrictamente la definición académica de Russell & Norvig: opera como un sistema de producción basado en reglas **Condición–Acción** evaluadas en orden de prioridad fija:

Prioridad	Regla	Condición (Percepción)	Acción Ejecutada
1	Captura	<code>move.captured !== undefined</code>	Ejecuta el salto de captura.
2	Coronación	Ficha normal alcanza fila 0 o 7.	Mueve hacia la casilla de coronación.
3	Evasión	<code>isPieceThreatened() === true</code>	Mueve la ficha amenazada para escapar.
4	Avance	Ficha normal que puede avanzar.	Prioriza la ficha más adelantada (<code>findAdvanceMove</code>).
5	Default	Ninguna condición anterior activa.	Ejecuta primer movimiento legal (<code>moves[0]</code>).

Limitaciones Deliberadas del Agente Reactivo:

1. **Ceguera hacia el futuro (Horizonte = 0):** Al intentar salvar una ficha amenazada (Regla 3), la mueve al primer destino legal disponible sin verificar si ese nuevo casillero también está amenazado.
2. **Vulnerabilidad a celadas / señuelos:** Si el jugador le ofrece una pieza como sacrificio para descolocarlo, el agente la comerá obligatoriamente por sus reglas y caerá en la trampa.
3. **Ausencia de función de utilidad global:** No calcula balances de piezas ni probabilidades de victoria; sus decisiones son puramente inmediatas.

5. Guía de Modificaciones Rápidas (Respuestas a Evaluadores)

Si los evaluadores o el docente solicitan ajustes en la lógica, aquí está exactamente dónde y cómo intervenir:

A. Si piden que la captura sea opcional (no obligatoria):

En `src/rules/CheckersRules.ts`, dentro de `getValidMoves()`, eliminar la condición `if (playerMustCapture) return captures;` y en su lugar retornar `[...captures, ...this.getNormalMoves(board, row, column)]`.

B. Si piden cambiar el perfil del agente (más defensivo o conservador):

En `src/agents/ReactiveAgent.ts`, dentro de `chooseMove()`, basta con intercambiar el orden de evaluación: colocar la **Regla 3 (Evasión)** antes de la **Regla 2 (Coronación)**, o priorizar fichas que permanezcan en bordes protectores.

C. Si piden cambiar la velocidad de juego de la IA:

En `src/game/scenes/GameScene.ts`, modificar los valores de 500 en `this.time.delayedCall(500, ...)` a 200 (ultrarrápido) o 1000 (pausado didáctico).

D. Si piden que las reinas vuelen a cualquier distancia (Damas Internacionales):

En `src/rules/CheckersRules.ts`, en `getNormalMoves` y `getCaptureMoves`, en vez de desplazarse solo 1 o 2 casillas fijas, iterar con un bucle `while` (`board.isInside(...)`) en cada diagonal hasta topar con obstáculo.

6. Cuadro Comparativo: Agente Reactivo vs. Agente por Objetivos (Minimax)

Criterio	Agente Reactivo Simple (S1)	Agente por Objetivos / Minimax (S2)
Filosofía	Reglas Condición–Acción (If-Then).	Búsqueda adversarial en árbol de estados.
Horizonte	Inmediato ($t = 0$). No predice.	Profundidad d ($t = 1, 2, \dots d$). Anticipa al rival.
Evaluación	Precedencia fija de reglas cualitativas.	Función heurística cuantitativa (material + posición).
Uso de Recursos	Tiempo $O(1)$, Memoria $O(1)$. Instantáneo.	Tiempo $O(b^d)$, requiere poda Alfa-Beta para optimizar.
Comportamiento	Mecánico, agresivo pero explotable.	Estratégico, precavido y capaz de tender trampas.